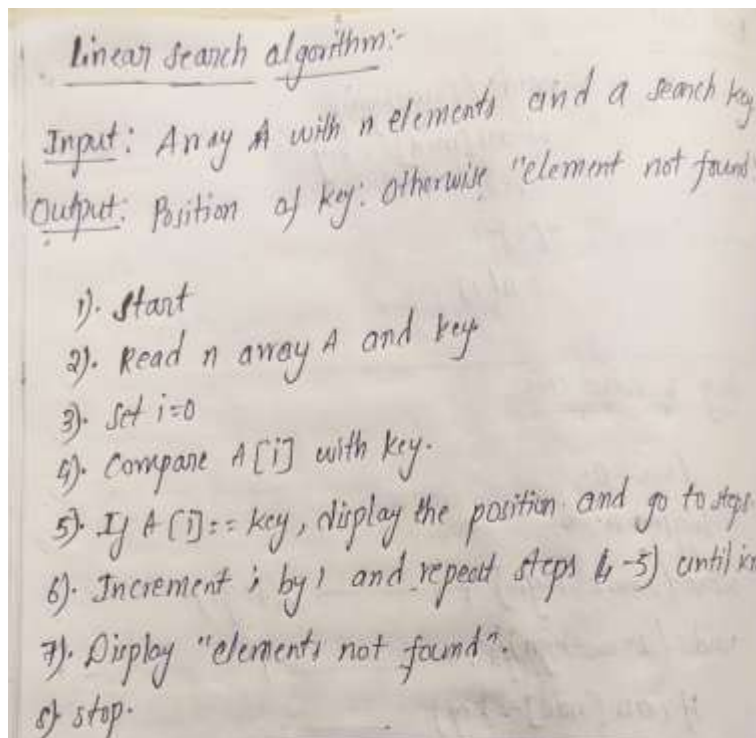


Practical: 2	Implementation and Time analysis of linear and binary search algorithm
05/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



Time Complexity:

Linear Search Time Complexity :

```
for(i=0; i<n; i++) {  $\rightarrow n-1$ 
    if(arr[i]==t) {  $\rightarrow 1$ 
        return i;  $\rightarrow 1$ 
    }
}
return -1  $\rightarrow 1$ 
```

$$2n + 2 + n + 1 + 1$$

$$= 3n + 4$$

$$T(n) = O(n)$$

Case:	Time Complexity:	Explanation
(i). Best	$O(1)$	element found at first position
(ii). Average	$O(n)$	element usually found somewhere in the middle.
(iii). Worst	$O(n)$	element found at last position (or) not present

Best case: $O(1)$

eg:- arr[i] = [10, 20, 30, 40, 50]

key $\rightarrow$ $T(n) = n/2$
 $T(n) = O(1)$

Avg case: element found at middle.

eg:- arr = [10, 20, 30, 40, 50]

key = 30, $\rightarrow n/2$ comparison
 $4/2 = 2$ arr[i]
 $T(n) = O(1)$

Worst case: $O(n)$

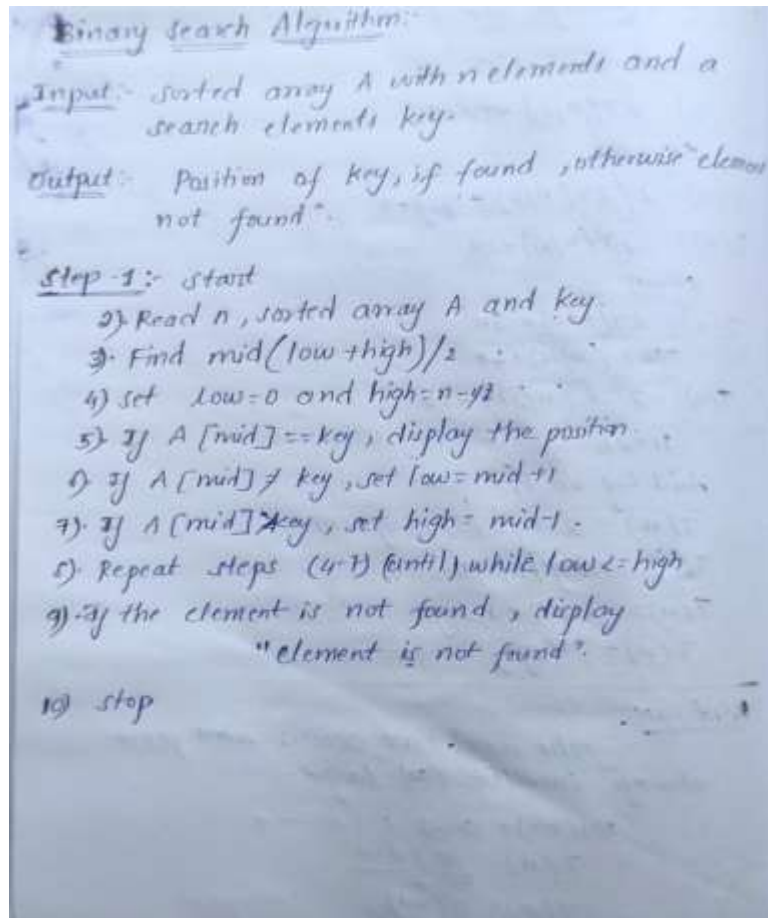
element found at last

eg:- arr = [10, 20, 30, 40, 50]

key = 50,
 $T(n) = n$
 $T(n) = O(n)$

Binary search

Algorithm:



Program code:

```
#include <iostream>
#include <vector>

using namespace std;

int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
```


FACULTY OF
TECHNOLOGY

Department of AI, ML & DS

01AI0506 | DAA Lab Manual

```
        low = mid + 1;
    } else { high =
        mid - 1;
    }
}
return -1;
}
```

```
int main()
```

```
{  
  
    int size;  
  
    cout << "Enter the number of elements: ";  
  
    cin >> size;  
  
    vector<int> arr(size);  
  
    cout << "Enter " << size << " elements in sorted order:\n";  
    for (int i = 0; i < size; i++) {  
        cin >> arr[i];  
    }  
  
    int target;  
  
    cout << "Enter the element to search for: ";  
  
    cin >> target;  
  
    int result = binarySearch(arr, target);  
  
    if (result != -1) {  
        cout << "Element found at index: " << result << endl;  
    } else {  
        cout << "Element not found in the array." << endl;  
    }  
}
```

Time complexity:

Binary search Time Complexity:

```

int binary(arr[], s, t) {
    l = 0;
    h = s - 1;
    while (l <= h) {
        m = (l + h) / 2;
        if (arr[m] == t) {
            return m;
        }
    }
}

```

$\Rightarrow m = m/2 + m/2 + n/3$
 $n/2 = 1, n = 24$
 $k = \log n$
 $T(n) = O(\log n)$

Best case $\rightarrow O(1)$
 $Mid = (low + high) / 2 \rightarrow 1$
 If $(a[mid]) = key \rightarrow 1$
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$

Avg case : $O(\log n)$
 $n \rightarrow n/2 + n/4 + n/8 + \dots$
 no. of division = $\log_2 n$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

Worst case:
 $n \rightarrow n/2 + n/4 + n/8 + \dots$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

Conclusion:

Linear search and binary search both are searching techniques used to find out an element to search in an array. Linear search checks the element one by one, works on both sorted and unsorted elements, then the elements of $O(n)$ and $O(n \log n)$ are the complexities.